

Lab 5 – Building A Movie Detail App With Navigation

1. Objective

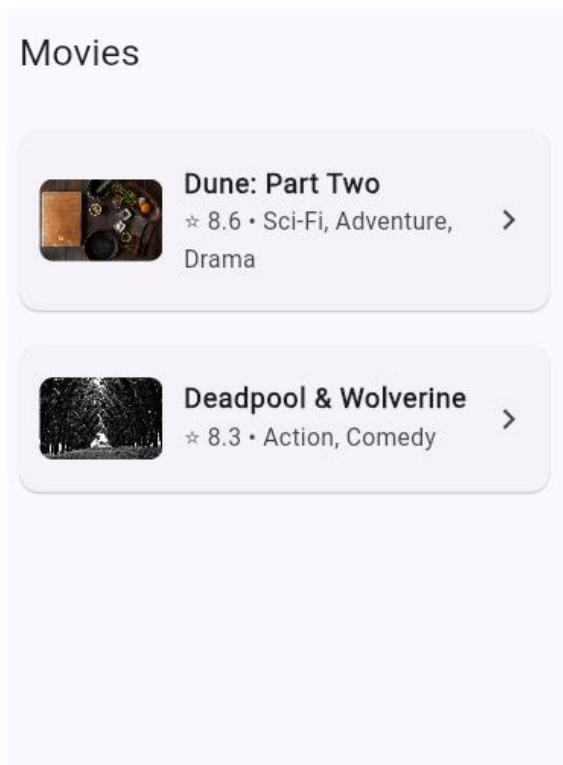
In this lab, students will design and implement a simple *Movie Detail App* that demonstrates how to use Flutter's navigation mechanisms (`Navigator.push`, `MaterialPageRoute`, or named routes) and basic UI composition.

The project reinforces the key ideas from **Module 5 – Navigation & State Management**.

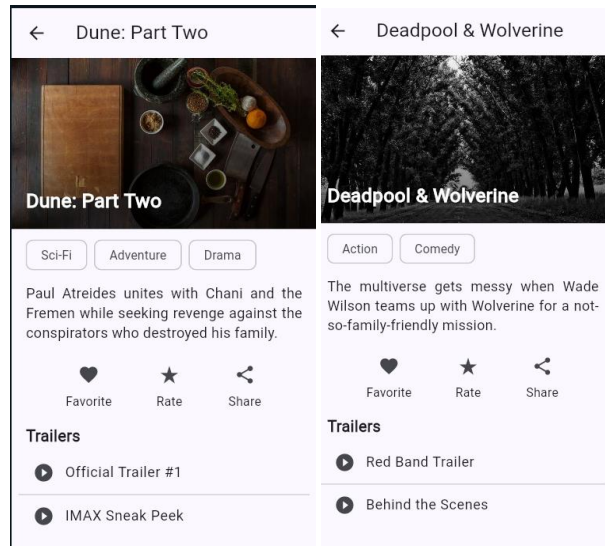
2. Requirements

Create a two-screen Flutter application:

- **Home Screen** – displays a scrollable list of movies (poster, title, and rating).



- **Movie Detail Screen** – shows the selected movie's information, including:
 - Poster (Hero banner with gradient)
 - Title and genres displayed as chips
 - Overview text
 - Action buttons (Favorite / Rate / Share)
 - List of trailers



Technical Requirements

1. Navigation must use either
 - `Navigator.push` + `MaterialPageRoute`, or
 - Named routes (`Navigator.pushNamed`).
2. Pass a **Movie** object between screens.
3. Layout should be scrollable and responsive.
4. Use static sample data (no API calls).
5. The app must run correctly in **DartPad** and **Android Studio / VS Code**.

Optional Enhancements

- Implement a *Favorite* toggle that updates state.
- Add a simple search bar to filter movies.
- (Advanced) Experiment with a *deep-link* that opens a specific movie detail page.

3. Guided Steps

Step 1 – Project Setup

Create a new Flutter project (flutter create movie_app) or open DartPad Flutter.

Step 2 – Define Data Model

Create a Movie class with fields for id, title, posterUrl, overview, genres, rating, and a list of Trailers. Add a sample_data.dart file containing 2–3 movie objects.

Step 3 – Build Home Screen

Use ListView.builder to display each movie card.

On tap → navigate to Movie Detail screen, passing the selected object.

Step 4 – Build Movie Detail Screen

Compose the UI progressively:

1. Blank Scaffold + AppBar
2. Hero Banner (Stack + Image.network + gradient)
3. Title & Genres (Column + Wrap + Chip)
4. Overview text with Padding
5. Row of IconButton (Favorite, Rate, Share)
6. Trailer list using ListView.builder

Step 5 – Test and Polish

Run the app → check navigation works and UI scrolls smoothly.

Optionally add state management with setState() or ChangeNotifier.

4. Expected Result

When finished, the app should:

- Display a movie list on Home.
- Navigate to a detail page with poster, genres, overview, actions, and trailers.
- Return to Home using the Back button.
- Match the Demo Progress sequence in **Figures 8.12 – 8.18** of *Mastering Flutter 2025*.

5. Submission

- **Deliverable:** Zipped Flutter project or DartPad share link.

- **Evaluation Criteria:**

- Correct navigation logic (30%)
- UI layout & design consistency (30%)
- Functionality & state handling (25%)
- Code clarity & comments (15%)